

CHAPTER 17

Testing

17.1. Space story

The first use of the Improved Electrothermal Hydrazine Thrusters (IMPEHTs) was on an operational mission. These thrusters had higher exhaust velocities due to a more powerful heater. As a consequence, the propellant feed tubes were narrower. The IMPEHTs could be off-pulsed. Thus the four thrusters on the North face of the spacecraft could be used for both North/South station keeping and attitude control during the burn. Off-pulsing was standard practice for East/West maneuvers that used standard hydrazine thrusters. This eliminated the need for hydrazine thrusters for roll/yaw attitude control. When the first maneuver started, the control performance was not as expected. It should have been very smooth. Instead, the spacecraft wobbled about. It was controlled but the attitude motion was greater than expected.

This launched an anomaly investigation. One path was to verify that the valves were behaving as expected. This test was done with a similar spacecraft in a high-bay. We did the test using PCs with LabView, the first time this had ever been done. LabView would send commands to the valves. The results were that the valves behaved as expected. Ultimately, it was determined that the problem was helium bubbles in the fuel. The new feed tubes were so narrow that bubbles could block the tube and hot helium did not produce any thrust. The bubbles were introduced to the fuel during pressurization. The propulsion system was a blowdown system that used helium as the pressurant.

17.2. A testing methodology

This section provides a discussion of testing of an attitude control system (ACS) and presents a test program necessary to ensure that the probability of failure of the attitude determination and control subsystem (ADACS) software due to hidden bugs is sufficiently low. This section also covers the testing of the flight software test-bed and supporting tools.

17.3. Reliability

The ACS must be able to meet the requirements for the spacecraft. That is, the system must be able to operate and meet all pointing requirements for the life of the spacecraft. Reliability will be a function of the reliability of the hardware and software. Hardware reliability is tied to the mean time between failures (MTBF). Most hardware follows a

bathtub curve for its probability of failure. The reliability is [1]

$$R(t) = e^{-\alpha t^\beta} \quad (17.1)$$

where

1. $\beta < 1$ Infant mortality;
2. $\beta = 1$ Useful life;
3. $\beta > 1$ End-of-life.

The failure rate is

$$Z(t) = \alpha \beta t^{\beta-1} \quad (17.2)$$

where α is the failure rate at $t = 0$. β changes, typically, at discrete times. To find the new α for the next segment, when β is different

$$\alpha(t(j)) = \frac{Z(j-1)}{\beta(t(j))t(j)^{\beta(t(j))-1}} \quad (17.3)$$

A typical bathtub curve is shown in Fig. 17.1. This type of curve applies to the hardware

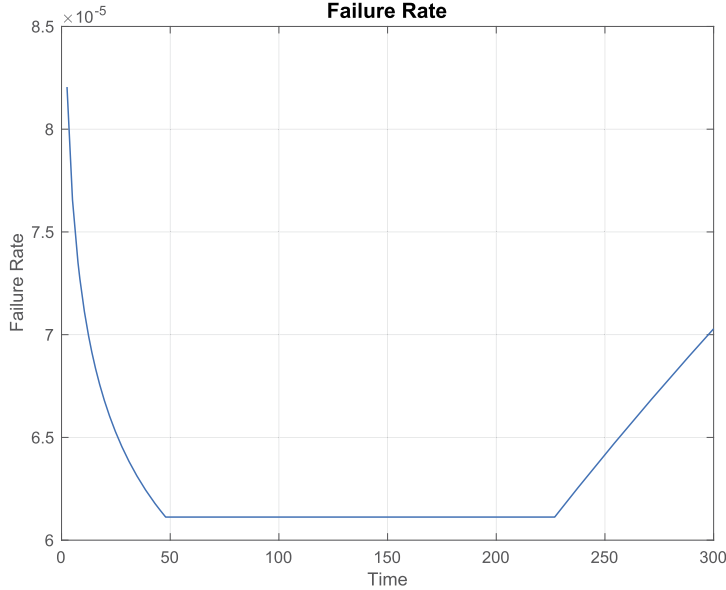


Figure 17.1 Bathtub curve. The end-of-life curve is shaped differently from the beginning-of-life curve.

in the ACS system. For parts in series the reliability is [2]

$$r = \prod_{k=1}^N R_k \quad (17.4)$$

For parts in parallel for a system where r must survive, where all R s are equal

$$R = \sum_{x=r}^N \binom{N}{x} R^x (1-R)^{N-x} \quad (17.5)$$

Infant mortality is dealt with by preflight testing. The flight portion is the bottom of the bathtub. The end-of-life is unavoidable. The reliability of the components must be sufficient that the system does not fail before its required lifetime.

17.3.1 Requirements flow and testing

Fig. 17.2 shows the requirements flow for flight software. The figure shows only the requirements levied by other subsystems that the flight software must support, such as the ADACS, power subsystem, and thermal subsystem. Requirements levied directly on the flight software are not shown.

Each subsystem has performance requirements that can only be met through operation of the flight software. This includes closed-loop attitude control, power management, etc. During the design cycle, each subsystem team performs analyses and simulations to demonstrate that the subsystem design meets its requirements. The subsystem then generates software requirements and these are realized in the flight software. Flight-software testing verifies that it meets the requirements levied on it by the subsystems, but not necessarily that the subsystem design, as implemented in flight software, meets the original subsystem requirements. In practice, this usually happens indirectly since flight software is often verified by comparing simulation results with the flight software to simulation results generated by subsystem simulations that have been shown to meet the performance requirement. The validity of this indirect approach is based on the assumption that the test platform for the flight software has sufficient fidelity to make the comparisons (in terms of performance) meaningful. In addition, the subset of tests run on the flight-software test platform are assumed to be representative of the subsystem tests originally used to demonstrate that the subsystem met its requirements. These assumptions may not be valid if the flight-software test platform simulation is of insufficient fidelity.

A careful distinction must be made among the types of requirements. Each subsystem has requirements it must fulfil and many fulfil their requirements through flight software. They do this by levying requirements on the flight software. In theory, if the flight software meets these requirements levied upon it by the subsystems, the subsystems will also meet their requirements. Unfortunately, this may not be the case and it is usually necessary to exercise the actual flight software and a subset of the subsystem hardware to demonstrate that the subsystem requirements have been met. Furthermore, some iteration between the flight-software-design phase and the subsystem-design phase is almost always necessary since subsystem requirements may not be implementable as first specified.

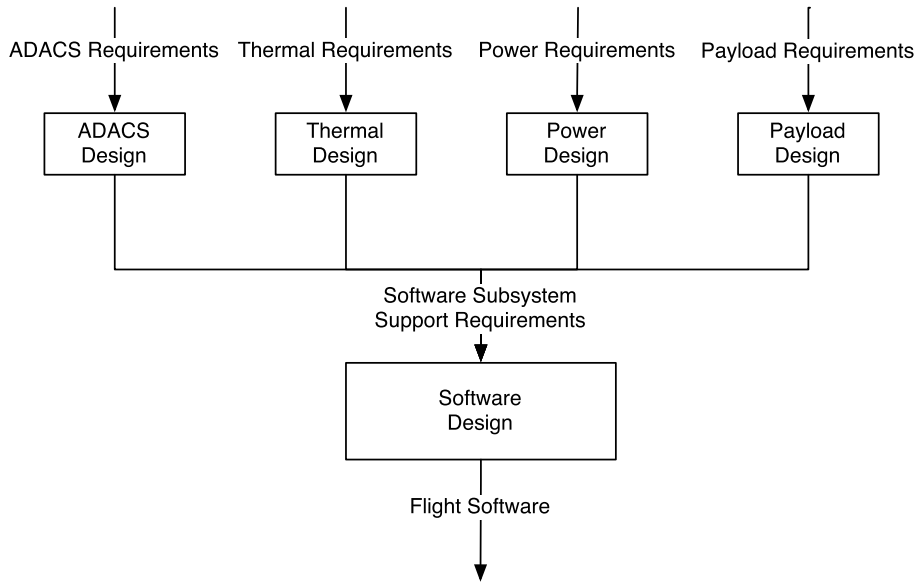


Figure 17.2 Typical requirements flow for an attitude control system.

17.3.2 Testing lifecycle for the ACS flight software

An example of a complete testing lifecycle for a flight-software product is given in Fig. 17.3. The rectangular symbols represent the design phases and the boxes with the rounded corners represent test methods or phases that are used to validate the results of a design phase before proceeding to the next phase. The first two phases in the design lifecycle involve the design of the ADACS algorithms, as opposed to the software that implements them in the flight computer. This phase is driven by the ADACS requirements. In the first phase, the ADACS algorithms are designed and tested in a control-analysis package. The design is then validated using a high-fidelity spacecraft simulation. In this case, the sensor, actuator, environment, and dynamics model are high fidelity. Flight computer arithmetic and interfaces are not modeled. The output of this phase is the ADACS specification and requirements documents along with supporting analysis memos.

The flight-software ADACS support requirements are derived from the results of the ADACS analysis. The remaining steps follow a standard software-design cycle. The test tools and platforms shown in the figure will be discussed in more detail later in this chapter.

The validation matrix lists requirements and the software modules that satisfy the requirements. A code walkthrough involves reading through the source code. Traditionally this is done by reading through a print out of the code or reading it in a text

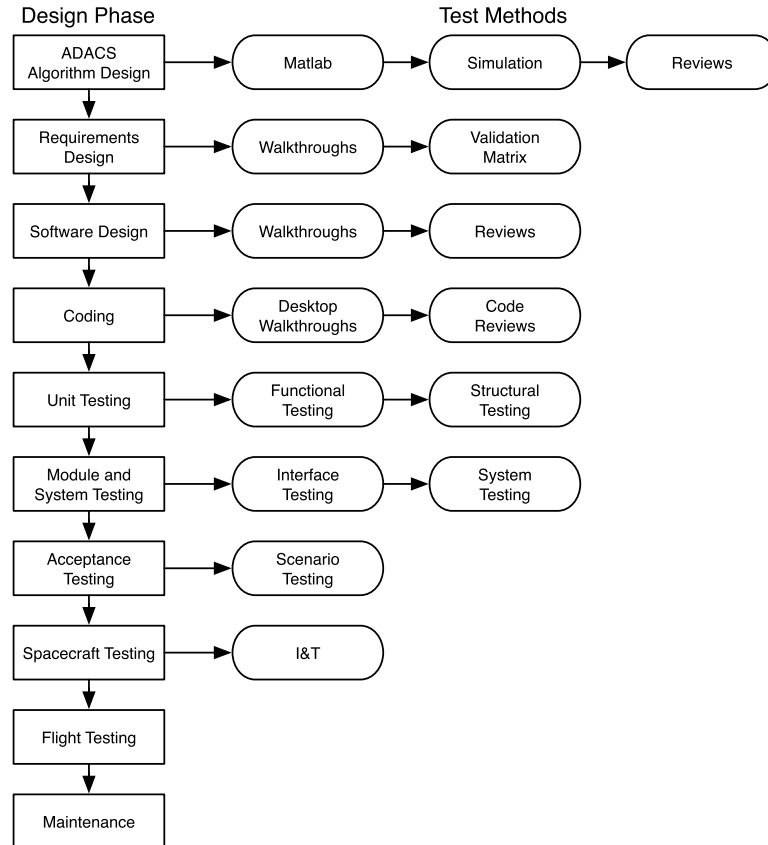


Figure 17.3 Testing lifecycle from algorithm design through flight.

editor. A variant is to walk through the code in a source-code debugger so that the flow of execution can be studied.

Integration and test (I&T) is where the spacecraft is assembled. The word “integration” is used because this stage involves the assembly of manufactured parts.

17.4. Flight-vehicle control-system testing

Flight-vehicle control-system testing goes through two stages. These are defined in Table 17.1. The actual flight vehicle is rarely available for test purposes until late in a program. Due to scheduling problems, it is rarely possible to perform extensive closed-loop testing on the spacecraft when it is in integration and test. Testing on the complete spacecraft is limited by the lack of internal visibility into the software. The only access to the flight software is usually through the command and telemetry interface. It is

difficult to gain access to internal variables in the flight software, making debugging difficult. Sensor inputs must be generated either by external targets, that provide little flexibility, or by inserting inputs into the lines running from the sensors to the flight computer. Measurements of the outputs are limited by whatever measurement channels are available on the actuators. For example, if the only speed measurement on a reaction wheel is the tachometer, it will not be possible to know the true reaction-wheel state. Mission-orbit operations, such as station keeping, are tested by performing the shortest

Table 17.1 Flight-vehicle-software testing.

Stage	Description
Preflight	All functions are tested either through closed-loop simulation or with open-loop test data (usually generated by another closed-loop simulation). Simulations may include some, none or all of the ADACS hardware. ADACS algorithms may be implemented in an abstract form, in another language (such as FORTRAN) in the target language, C, but on a different processor, and in the target language on the flight processor.
Flight	Transfer-orbit and acquisition ACS functions are tested as each is introduced in the mission sequence. Problems are usually resolved through procedural changes and work arounds. If necessary, the flight software can be patched.

possible version of the operation. Problems are usually resolved through procedural changes and work around. If necessary the flight software can be patched.

Simulating the complete space environment, including thermal, vacuum, and dynamics effects, is impractical with a complete spacecraft. In addition, it is difficult to simulate offnominal or worst-case conditions involving the actuators and sensors.

The most cost-effective solution is to run a simulation of the spacecraft dynamics and environment on another computer and use taps on the sensors and actuators to close the loop. This provides the most complete picture of how the flight software will perform, but must be used in conjunction with a high-visibility tool, such as a development station or processor emulator, to help trace problems that arise. Flight-software designers must rely on simulations of the vehicle and the operational environment to validate the flight software. Simulations are of five types, as given in Table 17.2.

Each of the simulations may include a complete closed-loop dynamics model of the spacecraft or may be used open loop with test data from another source usually from another simulation. Industry experience with different ADACS simulation testbeds is given in the next section.

17.5. Test levels (preflight)

This book uses the three levels of testing for the flight software. Unit testing is usually done by the designer of the procedure. It is meant to be exhaustive and guarantees

Table 17.2 Simulation types used to verify flight software.

Simulation	Description and Use
All software.	The vehicle, flight computer, and interfaces are modeled in software. This makes it a very flexible and portable tool. The fidelity of the simulation is limited by many factors including: 1. Available component data; 2. Host computer throughput; 3. Time available to design, build, and test numerical models. It is assumed that the fidelity of the simulation is sufficient to verify the performance of the algorithms in the flight software. Since the simulation can usually run many times faster than real-time, it is possible to test the long-duration behavior of the algorithms.
All software with flight-computer emulation.	The flight computer is emulated. This provides more accurate timing information at the expense of execution speed. It also gives excellent visibility into the workings of the software. Emulation can uncover compiler bugs or instruction-set problems.
All software with a flight computer in the loop.	This permits real-time operation. If dual-port memory is available the inner workings of the flight software can be monitored. However, if flight-prototype components are used, the visibility into the flight software is usually limited by the telemetry interface.
Software dynamics, all ADACS hardware in the loop but fixed base.	All interfaces are faithfully represented. The closed-loop simulation generates the stimuli for the sensors so sensor dynamics are not tested. This is sometimes done with a hardware breadboard ACS system. It can also be done on the actual spacecraft.
Software dynamics, all ACS hardware in the loop with a moving base.	All of the sensors are stimulated by sources representative of actual operation. For spacecraft with gyros this requires a multiaxis rate table. Earth and Sun targets are needed for the Earth and Sun sensors. Orientation of the rate table and Sun/Earth targets must be synchronized. This provides the highest possible fidelity, short of using the flight vehicle in operation. The cost is usually prohibitive for all but the most expensive satellites.

that the procedure will work correctly for all valid inputs. Since it is rarely possible to check every valid input, the inputs must at least encompass the full range of valid inputs. Boundary values must always be included in these tests. A combination of boundary values, random inputs (within the valid ranges), and “best guesses” of inputs that may cause problems, should be employed. Random inputs should be normally distributed about the mean input values. In addition, all logic paths must be exercised. The former type of testing is called black-box testing, the latter, in which test inputs are chosen to follow all logic paths, is known as white-box testing. Whenever possible, the procedure is tested against analytical results or, if that is not possible, against already verified, higher-fidelity models. Unit testing must be exhaustive because it is not feasible to fully test a procedure at the higher testing levels since it is difficult to guarantee, or prove, that

a given routine has been fully exercised. Unit testing is usually open loop, although closed-loop test drivers may be employed, see Table 17.3.

Table 17.3 Test levels.

Level	Scope	Tests
Unit	Individual procedures	Range of valid inputs. All paths and all code.
Module	Individual functions	All mission operations relevant to that module.
System	Entire ADACS	All mission sequences.

Module testing tests functions of the flight software. For example, the backup ephemeris is a function and is implemented by means of several different procedures and/or packages in the flight software. Module testing demonstrates that independent functions of the software work properly and that all of the procedures that comprise a module interact correctly. Module testing is usually open loop, although closed-loop test drivers may be used.

Module testing is the first step in verifying the interfaces between procedures. This is also accomplished during system testing. For some modules, such as thruster commanding, module testing would include testing of the hardware interfaces.

Most of the lower-level flight-software requirements will be validated using either unit or module-level testing, if only because the requirements are partitioned by function, and sometimes by module. Module testing should demonstrate that the module will meet its requirements over the full range of operating conditions. While unit testing should include all valid inputs, module testing need only include valid inputs within the operational range.

The highest level of testing, system testing, demonstrates that all of the modules work together correctly and that the flight software can perform all required mission operations. System testing should include realistic command inputs through the appropriate hardware.

All tests should employ a coverage-monitor tool. A coverage-monitor tool keeps track of what code is exercised in each test. For a given module, it is essential that all code be exercised in its module test. At the system level, all code should be exercised in at least one of the system-level tests. Studies have shown that testing done without coverage tools typically exercises only 55% of the code. In reality, it is impossible to actually test every path in anything but the simplest software and this must be factored into the software-development and quality-assurance processes.

17.6. Test levels (flight)

The first flight tests of the ADACS software will happen during actual mission operations in transfer orbit. Each function can be tested when it is first used. Whenever

possible, the duration of use of the function should be limited. Many spacecraft are first put into safe-mode prior to undergoing onorbit testing. This is rarely necessary if the software has been properly tested prior to flight. In many spacecraft it is necessary to precede immediately into transfer-orbit operations, so the safe-mode approach will not work. Consequently, it is desirable to develop a mission sequence that keeps the spacecraft progressing through mission phases without unnecessary risk to the spacecraft or mission.

17.7. Simulations

Simulations are a critical element in developing a spacecraft. It is not possible to test a spacecraft in the operational environment so simulations are required. For ADCS purposes the minimum simulation-state equations are

$$\dot{\omega} = I^{-1} (T - \omega^\times I \omega) \quad (17.6)$$

$$\dot{q} = f(\omega, q) \quad (17.7)$$

$$\dot{v} = F - \mu \frac{r}{|r|^3} \quad (17.8)$$

$$\dot{r} = v \quad (17.9)$$

where F and T are functions of external disturbances and external controls such as thrusters and magnetic torquers. The external disturbances will be a function of r and v thus it is important to simulate the orbital dynamics along with the attitude dynamics. If the spacecraft were a gyrost, additional equations for the reaction wheels would be needed. This dynamical model is sufficient for design of an ADCS of a rigid-body spacecraft.

When you first build a simulation, use ideal sensor and actuator models. This allows you to verify your control laws. The next step is to accurately model sensors and actuators and test with the detailed models to make certain the performance does not change and to identify any issues due to the sensors and actuators.

Occasionally, moving-base simulators and air-bearing simulators are used. While sometimes Sun and sky targets are used, often the sensors have an input that allows you to produce an output synchronized with the simulation. Given the sophistication of software models, hardware simulations are rarely needed, except to verify interfaces. This is usually done routinely during integration and test.

For operations and training, large-scale simulations are often developed. It is rarely necessary to use more complex ADCS models, but it becomes necessary to add other subsystems such as power, thermal, and communications. While ACS engineers always work with dynamical models, other subsystems do not. For this reason, ACS engineers are often called upon to help other subsystems add their models to this type of simulation.

17.8. Software-development standards

Many formal standards exist for software development, both commercial and government. Some of the most popular (or most used) are listed in Table 17.4.

Table 17.4 Software-development standards.

Standard	Description
MIL-STD-2167A	Defense system-software development. Covers the acquisition, development, and support of software systems.
NASA-STD-8719.13A	Expands on the requirements of NMI 2410.10, “NASA Software Management Assurance and Engineering Policy,” NHB 1700.1(V1), “NASA Safety Policy and Requirements Document”; and NASA-STD-2201-93, “Software Assurance Standard.”
CMMI	Capability Maturity Model Integration (CMMI) is an approach to process improvement that is used to set up effective software-development systems.
IEEE	The Institute for Electrical and Electronics Engineers has developed a comprehensive set of software-development standards that are used throughout industry.
UML	Unified Modeling Language (UML) that is implemented using the IBM Rational Rose products provides a structure for development of large software projects from requirements definition through test and evaluation.
ECSS-E-40	European Space Agency standards for software engineering.
ECSS-Q-80	European Space Agency standards for software product assurance.

References

- [1] B. Seymour, BMTTF, Failrate, Reliability and Life Testing.
- [2] Jeremy Kasidin, Reliability, Princeton University Mechanical and Aerospace Engineering Course Notes..